咱们先把话说清楚。氛围编程（Vibe Coding）本身没问题，问题在你。

你听说可以跟 AI 编程助手聊天就能交付软件，就觉得自己成了魔术师。于是你打开 AI，用一句话描述了你的应用想法，然后期待魔法发生。但出乎意料的是，你得到的是一堆屎山、幻觉出来的 UI 和颜色、页面路由不通、还有一个"勉强能用"直到彻底崩掉的 App。

然后你怪 AI 犯了错。

像个傻子一样。

真相是：AI 产生幻觉不是因为它坏了，而是因为你没给它任何可以依靠的东西。

没有结构、没有清晰度、没有基础。

氛围编程是有效的。

但前提是你得理解自己在构建什么，并给 AI 代理一个真正全面的系统来工作。 接下来是你要知道的一切：构建模块、词汇表、真正的工作流程，简单到原始人都能看懂。

如果你读完这篇还是做不出东西，那问题是努力程度，不是信息不足。

这不是一篇扫一眼就忘的推文。这是完整的系统，从头到尾的氛围编程指南。现在收藏它，保存到你 Clawdbot 的记忆里。

每次开始新项目都回来看。把这篇当参考手册的人会构建出惊人的东西。扫一眼就忘的人会继续卡壳和崩溃。

现在，让我们来修复你。

一、为什么你失败了

你失败是因为你跳过了基础。

你不知道什么是组件。你不知道状态是什么意思。你不理解为什么点击按钮没反应。你不知道为什么在你的笔记本上看起来很好，但在手机上就崩了。

正因为不知道这些，你无法向 AI 描述它们。

AI 是翻译器。 它把你的意图转换成代码。但如果你的意图是屎，代码也会是屎。如果你无法清晰表达想要什么，AI 就会猜。而猜测会叠加成彻底的混乱。

修复方法不是更好的提示词。
修复方法是更好的理解。

一旦你知道自己在构建什么，提示词就变得微不足道。话语自然流露，因为你终于知道该问什么了。

二、文档优先系统

这是所有人都做错的地方。

你打开 Cursor，打开聊天，开始描述你的 App，让 AI 立即开始编码。没有计划、没有参考、没有真相来源。

这就是你的项目在前几个文件开始后就崩溃的原因。

真正的系统是：文档第一，代码第二。永远这样。

在写任何一行代码之前，你应该先写项目的规范文档 Markdown 文件。

清晰、具体、明确的描述，说明你在构建什么。

为什么?

因为 AI 编程工具能力高但确定性低。它们在没有结构护栏的情况下执行任务。缺乏锁定约束和权威文档会导致 AI 幻觉需求、做出未经授权的架构决策，并产生解决你从未明确表述过的问题的代码。

失败模式不是缺乏编码能力。
失败模式是缺乏纪律和上下文保存。

你应该在碰任何代码之前写文档栈

六份规范文档定义你的整个项目，再加上一些让 AI 保持一致并在会话间保持持久的文件。

规范文档（你的知识库）：

1. PRD.md（产品需求文档） - 完整规格。你在构建什么、为谁构建、有什么功能、什么在范围内、什么明确在范围外。用户故事、成功标准、非目标，以及每个功能的具体标准。这是你的合同。AI 读了这个就知道"完成"对你来说是什么样子。没有这个？你不是在构建 App，你是在祈祷一个 App 出现。

2. APP_FLOW.md - 每个页面和每个用户导航路径都用 plain English 记录。什么触发每个流程。逐步序列与决策点、成功时发生什么、错误时发生什么，以及屏幕清单与路由。这防止 AI 猜测用户如何在 App 中移动。

3. TECH_STACK.md - 每个包、依赖、API 和工具都锁定到确切版本。没有歧义。当 AI 看到"使用 React"，它可能选任何版本。当它看到"Next.js 14.1.0, React 18.2.0, TypeScript 5.3.3"，它构建的完全是你指定的。这份文档消除幻觉依赖和随机技术选择。

4. FRONTEND_GUIDELINES.md - 你的完整设计系统。字体、带确切十六进制代码的调色板、间距刻度、布局规则、组件样式、响应式断点，以及UI 库偏好。每个视觉决策都锁定。AI 参考这个来创建每个组件。不再有随机颜色或不一致的间距。

5. BACKEND_STRUCTURE.md - 数据库模式，每张表、每列、类型和关系都定义好。认证逻辑、API 端点合约、存储规则，以及边缘情况。如果你用 Supabase，这份文档包含确切的 SQL 结构。AI 根据这个蓝图构建你的后端，而不是根据它自己的假设。

6. IMPLEMENTATION_PLAN.md - 逐步构建序列。不是"构建 App"。更像是：步骤 1.1 初始化项目，步骤 1.2 从 TECH_STACK.md 安装依赖，步骤 1.3 创建文件夹结构，步骤 2.1 按照 FRONTEND_GUIDELINES.md 构建导航栏组件，等等。

步骤越多，AI 猜测越少。AI 猜测越少，幻觉越少。

这些文档相互引用。PRD 定义功能，APP FLOW 定义用户如何体验它们，TECH STACK 定义用什么构建它们，FRONTEND GUIDELINES 定义它们长什么样，BACKEND STRUCTURE 定义数据如何工作，IMPLEMENTATION PLAN 定义构建顺序。

这是你的知识库。AI 会读这些并得到它所需的一切。

两份会话文件（你的持久层）：

CLAUDE.md - 这是 AI 每次会话自动首先读取的文件。它包含每个 AI 会话必须遵循的规则、约束、模式和上下文。你的技术栈摘要、文件命名约定、组件模式、设计系统令牌。它是允许的和禁止的。把它当作 AI 针对你特定项目的操作手册。Claude Code 可以从项目根目录读取这个，甚至不需要你要求。

progress.txt - 这是所有人都漏掉的文件。这个文件跟踪已完成的内容、进行中的内容，以及接下来的内容。每次你完成一个功能，你更新这个文件。每次你开始新会话，每次你打开新终端窗口，每次你切换分支，AI 首先读取这个文件来获取你进度的上下文记忆。没有它，每个新会话都从零上下文开始，伴随着一大堆错误。有了它，AI 精确地从你离开的地方继续。

这就是为什么这很重要：AI 在会话间没有记忆。 当你关闭终端、打开新终端，或开始新聊天时，一切都消失了。progress.txt 是你的外部记忆。它是会话之间的桥梁。

虔诚地更新它。在每次实现完成功能后，详细记录构建了什么、什么有效、什么坏了、接下来做什么。

三、 interrogation 系统

在你写文档之前，让 AI 把你的想法撕碎。

这是改变一切的提示词：

"在写任何代码之前，在 Planning 模式下无尽地审问我的想法。不要假设任何问题。问问题直到没有假设剩下。"

AI 在你的清晰度结束时产生幻觉。所以如果你延伸你的清晰度，你会迫使 AI 在你开始在一个破碎的基础上构建之前找到你思维中的 gaps。

AI 应该问你什么：
这是给谁用的？用户采取的核心行动是什么？他们完成那个行动后发生什么？需要保存什么数据？需要展示什么数据？错误时发生什么？成功时发生什么？这需要登录吗？这需要数据库吗？需要在手机上工作吗？

如果你不能回答这些问题，你还没准备好构建。先去回答它们。

这就是好的 interrogation 看起来的样子。假设你在构建一个食谱分享 App：

这是给谁用的？想要保存和分享食谱的家庭厨师。核心行动？用户用标题、食材清单、步骤和照片创建食谱。之后发生什么？食谱保存到他们的个人资料并显示在公共 feed 上。保存什么数据？食谱标题、食材（数组）、步骤（数组）、照片 URL、作者 ID、时间戳。展示什么数据？最近食谱的 feed、单个食谱详情页面、用户自己的食谱集合。错误状态？上传失败、缺少必填字段、未授权编辑尝试。需要登录？是的，要创建食谱。浏览是公开的。数据库？是的，users 表和 recipes 表带外键关系。手机？是的，大多数用户会在做饭时用手机添加食谱。

这些答案不只是答案。它们是你规范 Markdown 文档的原材料。用户描述喂给你的 PRD。数据结构喂给你的 BACKEND STRUCTURE。流程喂给你的 APP FLOW。手机需求喂给你的 FRONTEND GUIDELINES。

一旦 interrogation 完成，你对一切都有清晰答案，使用第二个提示词：

"基于我们的审问，生成我的规范文档文件：PRD.md、APP_FLOW.md、TECH_STACK.md、FRONTEND_GUIDELINES.md、BACKEND_STRUCTURE.md、IMPLEMENTATION_PLAN.md。用我们对话中的答案作为素材。要具体且详尽。没有歧义。"

AI 会从审问输出中起草所有文档。你审查它们，纠正任何模糊的，添加任何缺失的，然后把它们锁定为你的真相来源。

顺序：审问 → 文档 → 代码。

永远不要跳过这些步骤。

四、UI vs UX

两个每个人都在乱用的术语，但没人简单解释过。

UI 是它看起来的样子（颜色、字体、按钮形状、间距），也就是视觉层。

UX 是使用起来的感觉（有人能弄清楚要做什么吗？流程直观吗？人们会卡住吗？是沮丧还是愉悦？）

你可以有漂亮的 UI 和糟糕的 UX。漂亮的按钮但没人知道怎么点。你可以有丑陋的 UI 和很棒的 UX。功能性的、清晰的，人们确切知道要做什么，但它很丑。

当你跟 AI 说话时，明确你指的是哪一个。"让它看起来更好"是 UI。"让它更容易用"是 UX。"让按钮更明显"是两者都有。

这里有个大多数人不知道的技巧：用截图作为参考。找到你喜欢的 App 或网站（可能在 Framer 或 Dribbble 上），截图，直接喂给 Claude Code 或 Cursor。提示"匹配这个布局"、"复现这个确切的 UI"、或"用这个 UI 作为灵感"。AI 可以分析图片并从中提取设计模式、间距、调色板、排版和组件结构。这比用文字描述设计要好无限倍。Google 的 Gemini 3 Pro 是目前我发现的设计匹配高级美学的最佳模型。

你也可以截图你自己的正在进行的作品，说"这是它现在的样子，这是问题，修复它"。视觉参考比任何书面描述更快地消除歧义。

更好的是：Cursor 有个浏览器侧边栏让你可以同时设计和编码。你可以实时看到你的 App 渲染，点击元素、移动东西、更新颜色、测试布局、实时试验 CSS，然后通过 Agent 模式直接把这些更改应用到你的代码库。这是迭代视觉设计最快的方式，不需要截图、粘贴、等待的循环。

你需要知道的设计风格

这些是目前主导的视觉语言。当你在 FRONTEND_GUIDELINES.md 或给 AI 的提示词中引用这些术语时，这些术语解锁具体的、可识别的美学，而不是模糊的描述。

Glassmorphism（玻璃拟态） - 磨砂玻璃效果。带背景模糊的半透明元素、微妙的边框、柔和阴影漂浮在彩色背景上。想想 Apple 的 macOS、Windows 11、Spotify 的移动 App。在 CSS 中使用 `backdrop-filter: blur()`。创造深度和层次而不需要厚重阴影。看起来高级。非常适合卡片、模态框、导航栏和仪表盘。风险：透明度可能降低可读性，所以保持文本对比度高。

Neobrutalism（新粗野主义） - 原始、大胆、故意不精致。高对比度颜色、厚黑边框、平阴影、冲突的调色板、古怪字体。想想 Gumroad、早期 Notion 的感觉。是极简主义带态度。非常适合创意品牌、作品集、独立工具。很突出因为其他东西看起来都一样。告诉 AI："新粗野主义风格，厚边框和粗体原色。"

Neumorphism（软 UI） - 元素看起来像是从背景中挤出或压入。两边柔和的漫射阴影创造触觉的 3D 感觉。微妙优雅但可访问性棘手，因为低对比度可能让按钮难以区分。最适合小 UI 元素、切换、滑块、卡片。不适合作为你的整个设计语言。

Bento grid（便当网格） - 模块化布局，内容排列在不同大小的块中，像日本便当盒。Apple 推广了这个。不同尺寸的卡片创造视觉节奏和层次。大卡片放重要内容，小卡片放次要信息。响应式本质，因为网格在手机上会重新排列。非常适合仪表盘、产品页面、功能展示、作品集。这可能是目前最实用的趋势，因为它解决了真正的布局问题。

Dark mode（暗黑模式） - 不再只是偏好切换，它是一个设计系统。深色背景配浅色文本、仔细的对比度比例、柔和的强调色。减少眼睛疲劳、在 OLED 屏幕上省电、看起来高级。如果你在做任何消费级 App，从一开始就规划好亮色和暗黑模式。不要之后随机添加。在 FRONTEND_GUIDELINES.md 中定义好两个调色板和主题。

Kinetic typography（动态排版） - 移动、拉伸、响应滚动或光标的文本。进入时动画的标题、滚动时缩放的文本、交互式字体处理。不只是淡入。用现代 CSS 和 Framer Motion 等库，这可以在没有繁重 JavaScript 的情况下实现。在适用的情况下，用于 hero 区域和关键时刻。

Micro-interactions（微交互） - 响应用户操作的小动画。悬停时微妙缩放的按钮。点击时弹跳的复选框。感觉活着的加载旋转器。这些小细节把抛光产品和真正业余的区分开来。它们传达界面是响应的和活着的、高级的。Framer Motion 和 CSS 过渡轻松处理这些。

当你给 AI 提示词时，不要说"让它看起来现代"。说"玻璃拟态卡片配便当网格布局、暗黑模式、和悬停状态的微交互"。这是一个具体的、可构建的方向。

在编码开始前把设计决策锁定在 FRONTEND_GUIDELINES.md 中。挑选这些风格中的一两种，定义你的调色板、间距刻度、圆角、阴影值和动画时间，AI 会在它们有扎实文档的情况下一致地遵循它们。如果没有文档，每个组件都会看起来不同，会零一致性。

五、组件

组件是可复用的界面片段。

想象乐高。每块砖是一个组件。你把它们加在一起构建东西。

按钮是一个组件。导航栏是一个组件。显示产品的卡片是一个组件。表单是由小组件（输入框、标签、按钮）组成的组件。

为什么这对氛围编程很重要：

当你说"给我构建一个落地页"，AI 必须决定创建什么组件。如果你不指定，它猜测。它可能把所有东西建成一团糟，而不是干净、可复用的片段。

更好的提示词：
"用这些组件构建落地页：导航栏、hero 区域、功能网格（3 张卡片）、推荐轮播、CTA 区域、页脚。"

现在 AI 确切知道要创建什么片段。每个片段是隔离的。每个片段可以独立编辑。这就是组件化思维的力量。

六、布局

布局是东西在页面上的位置。

每个网站都是盒子套盒子。掌握这个概念...你会理解 90% 的网页设计。

主要盒子：顶部的 header/导航栏、中间的主要内容、旁边的可选侧边栏、底部的页脚。

在主要内容里面，更多盒子：分割页面的 sections、控制宽度的 containers、把物品组织成行和列的 grids、分组相关内容的 cards。

当你向 AI 描述布局时：
"两列布局。左边侧边栏，250px 宽。主要内容占据剩余空间。侧边栏固定，不滚动。"

这是完整的布局指令，现在 AI 确切知道要构建什么。

七、State（状态）

State 是变化的数据。

当你点击按钮发生事情时，state 变了。当你在表单打字并看到你的文本出现时，state 变了。当你切换暗黑模式颜色翻转时，state 变了。

State 是你的 App 感觉活着的原因。没有 state，一切都是静态的。什么都不应该响应。

常见的 state 例子：菜单是打开还是关闭？用户是登录还是登出？购物车里有什么物品？输入框里有什么文本？这是在加载还是完成了？这是成功还是失败？

当你的按钮没反应时，通常是 state 问题。点击发生了，但没什么告诉 App 要更新。

当你跟 AI 谈交互性时：
"当用户点击这个按钮，把 modal state 设为打开。当他们点击 modal 外面，设为关闭。"

现在 AI 知道要跟踪什么 state 以及何时改变它。

八、Styling（样式）

样式是你让东西好看的方法。

CSS 是控制外观的语言。颜色、间距、字体、大小、位置。

Tailwind 是一个快捷系统。不写 CSS 文件，你直接给元素加类。`bg-blue-500` 让背景变蓝。`text-xl` 让文本更大。`p-4` 添加内边距。

Design tokens（设计令牌）是你能复用的一致值。你的品牌蓝不只是任何蓝色，它在到处都是相同的十六进制代码。你的间距不是随机的，它总是 4px 的倍数。你的圆角在每个卡片上都是一样的。你的阴影值在每个抬升元素上都是一样的。使用设计令牌帮助最小化硬编码颜色和 UI 幻觉。

这就是区分业余 App 和精致 App 的东西。当每个组件使用相同的设计令牌，你的 App 感觉有凝聚力。当每个组件发明自己的值，它看起来像五个人建的。

在开始编码前把这一切都锁定在 FRONTEND_GUIDELINES.md 中：

带确切十六进制代码的调色板，用于主色、辅色、背景、表面、文本、边框、成功、错误和警告。你的间距刻度（4px, 8px, 12px, 16px, 24px, 32px, 48px, 64px）。你的字体栈，包括标题、正文和小文本的大小。你的圆角值。你的阴影定义。你的过渡时间。

当 AI 有这份文档，它生成的每个组件都匹配。没有它，你会花几个小时手动修复本不该存在的不一致。

相信我，我去过那里，那是个噩梦。避免痛苦。

你的样式指令越具体，AI 猜测越少。"用 16px 内边距和 8px 圆角把背景做成 #3B82F6" 每次都能打败"做成蓝色带一些内边距"。

九、Responsive Design（响应式设计）

Responsive 意味着你的网站在所有屏幕尺寸上都工作。

你的笔记本很宽。你的手机很窄。同一个网站，不同的布局。

在大屏幕上，显示更多列。在小屏幕上，垂直堆叠东西。在大屏幕上，显示完整导航。在小屏幕上，显示汉堡菜单。

Breakpoints 是设计变化的屏幕宽度。手机是 0-640px。平板是 640-1024px。桌面是 1024px 以上。

Mobile-first 意味着你先为最小屏幕设计，然后为大屏幕添加复杂度。这是 Tailwind 默认的工作方式。无前缀的样式应用于所有屏幕。带前缀的样式如 `md:` 和 `lg:` 只在这些断点触发。所以 `flex flex-col md:flex-row` 意味着手机上堆叠，平板及以上并排。

这不是偏好，这是策略。Mobile-first 迫使你优先考虑内容和极简主义。小屏幕上绝对必须有什么？那是你的核心。其他一切都是大屏幕的渐进增强。当你被限制在更小的尺寸时，它也推动你确保用户体验是简单、极简、真正性感的。

在 FRONTEND_GUIDELINES.md 中定义你的断点和响应模式。"导航：768px 以下汉堡菜单，以上完整水平导航。网格：手机单列、平板两列、桌面三列。字体大小：每个断点放大 15%。"当 AI 有这些规则文档化，它构建的每个组件都正确响应每种屏幕尺寸，不需要你重复指令。

总是先考虑手机。50%+ 的用户可能在手机上。如果你只在笔记本上测试，你会为大多数人交付破碎的体验。

十、Pages vs Routes

Page 是用户看到的。Route 是显示那个 page 的 URL。

`yoursite.com/` 显示首页。`yoursite.com/about` 显示关于页面。`yoursite.com/products/123` 显示产品 123。

Routes 可以是静态的（`/about` 总是显示相同页面）或动态的（`/products/[id]` 基于 id 显示不同产品）。

当你向 AI 描述你的 App 时：
"我需要 4 个页面：首页（/）、关于（/about）、所有产品（/products）、单个产品（/products/[id]）"

现在 AI 知道完整结构。它构建正确链接的导航，并在正确位置创建正确的文件。

十一、Frontend vs Backend

Frontend 是用户看到和交互的。在他们的浏览器里运行的界面。

Backend 是幕后发生的。数据库、用户账户、处理，以及在服务器上运行的东西。

当你提交表单时：frontend 收集你的输入，发送到服务器，服务器验证并保存，服务器发回确认，frontend 显示成功消息。

对于简单网站你可能不需要 backend。静态页面、没有数据库、没有账户。对于有用户、保存数据或复杂逻辑的 App，你需要两者。

upfront 告诉 AI 你需要哪个。"这只是 frontend，只是一个落地页"和"这需要带用户账户和数据库的 backend"产生完全不同的输出。

十二、APIs

API 是两个系统相互交谈的方式。

你的 frontend 需要从 backend 获取数据。它通过 API 询问。所以如果你的 App 需要天气数据，它通过他们的 API 询问天气服务。如果你的 App 需要处理支付，它通过 API 与 Stripe 交谈。

把 API 当作服务员。你告诉服务员你想要什么。服务员去厨房带回你的订单。

常见模式：GET 检索数据，POST 发送数据，PUT 更新数据，DELETE 删除数据。

当你跟 AI 交谈时：
"当页面加载时，向 /api/products 发起 GET 请求并在网格中显示结果。"

现在 AI 确切知道要构建什么。

十三、Databases

数据库是你永久保存东西的地方。没有数据库，刷新时一切都重置。用户注册，刷新，没了。物品加入购物车，刷新，没了。

初学者常见选项：
Supabase 是最简单的开始。免费层、好文档、也处理认证。不确定用什么？用 Supabase。

什么时候需要数据库？用户可以创建账户。用户可以保存自己的内容。你需要存储提交。你有随时间增长的数据。

什么时候不需要？静态网站，每个人相同内容。没有用户交互的作品集。只通过表单服务收集邮箱的落地页。

当跟 AI 交谈时：
"用 Supabase。我需要一个带 email 和密码的 users 表，以及一个带 title、content 和 user_id 的 posts 表。"

现在 AI 知道你的数据结构，可以构建一切来匹配。

十四、Authentication

Authentication 是登录/登出，也就是证明某人是谁。

这比看起来难。密码需要加密、会话需要管理、令牌需要处理。很容易搞砸。

不要从零构建 auth。用服务。Clerk 是最简单的，开箱即有很棒的 UI。如果你已经在用 Supabase 做数据库，Supabase Auth 可以工作。

当跟 AI 交谈时的例子：
"用 Clerk 做认证。用户可以用 email 或 Google 注册。登录后，重定向到仪表盘。"

让服务处理困难的部分，你可以直接插进去。

十五、File Types

你会在代码库中看到很多文件。这些实际上是它们中的一些：

- `.html` 是网页的结构
- `.css` 是样式规则
- `.js` 是让东西可交互的 JavaScript
- `.jsx` 是带 HTML 类似语法的 React JavaScript
- `.ts` 是 TypeScript，带类型的 JavaScript 能捕获错误
- `.tsx` 是带 React 的 TypeScript
- `.json` 是结构化数据
- `.md` 是 markdown，格式化文本

- `.env` 是环境变量和密钥
- `.gitignore` 告诉 git 跳过哪些文件

重要的一项：`.env`。这是你放 API 密钥、密钥、密码的地方。永远不要分享这个文件。永远不要提交到 git。永远不要截图。如果你泄露了你的 .env，你就泄露了对一切的访问，一些幸运的家伙现在会帮你跑几千美元的 API 账单。别犯那个错误。

十六、Folder Structure

你的文件放在哪里很重要。

混乱的项目让 AI 困惑。

标准结构：

```
my-app/
├── src/
│   ├── app/            → 页面和路由
│   ├── components/     → 可复用 UI 片段
│   ├── lib/            → 工具、辅助函数
│   └── styles/         → CSS 文件
├── public/             → 图片、静态文件
├── .env                → 密钥（永不分享）
├── CLAUDE.md           → AI 规则和上下文
├── progress.txt        → 会话跟踪
├── PRD.md              → 产品需求
├── APP_FLOW.md         → 用户流程和导航
├── TECH_STACK.md       → 锁定的依赖
├── FRONTEND_GUIDELINES.md → 设计系统
├── BACKEND_STRUCTURE.md   → 数据库和 API 规格
├── IMPLEMENTATION_PLAN.md → 逐步构建顺序
├── package.json        → 依赖
└── README.md           → 项目概览
```

当 AI 生成代码时，告诉它把代码放在哪里。"在 src/components/Button.tsx 创建按钮组件。"

如果你不指定，AI 可能把你的文件放在任何地方，没什么可能正确连接。

十七、实践中的文档系统

Markdown 不是可选的。它是 AI 思考的语言。

你写的每个 .md 文件都成为 AI 可以读取、理解和遵循的参考文档。这就是文档优先方法有效的原因。你不是为人类写文档。你是在为 AI 写约束。

这是每份规范 Markdown 文档在构建期间的使用方式：

PRD.md 是你范围的事实来源。当 AI 试图添加你没要求的功能时，你指向 PRD。"只构建 PRD.md 中的内容。"当你决定接下来构建什么时，你检查 PRD。它在开始之前就杀死了范围蔓延。

APP_FLOW.md 是你构建页面转换和用户旅程时参考的。"完全按照 APP_FLOW.md 第 3 节记录的那样构建登录流程。"AI 现在知道那个流程中的每个屏幕、每个重定向、每个错误状态。

TECH_STACK.md 在项目初始化和 AI 试图引入新依赖时被引用。"只使用 TECH_STACK.md 中列出的包。没有询问不要添加新依赖。"这防止 AI 随机导入你没批准的库。

FRONTEND_GUIDELINES.md 是 AI 应该为每个创建的组件引用的文档。你的颜色、间距、排版、组件模式、响应规则。"按照 FRONTEND_GUIDELINES.md 第 2 节样式化这个组件。"每个文件一致的设计。

BACKEND_STRUCTURE.md 定义你的数据层。"按照 BACKEND_STRUCTURE.md 第 2.1 节创建 users 表。"AI 构建你指定的确切模式，不是它自己的解释。

IMPLEMENTATION_PLAN.md 是你的执行序列。"我们在 IMPLEMENTATION_PLAN.md 的步骤 4.2。只构建这个步骤。"这防止 AI 跳到前面或乱序构建东西。

CLAUDE.md 是主配置文件。它坐在你的项目根目录，Claude Code 每次会话自动读取它。它包含所有六份文档的浓缩规则：技术栈摘要、命名约定、文件结构、组件模式、禁止的操作。把它当作 AI 为自己加载的操作系统手册。

这里大多数人漏掉的一招：CLAUDE.md 是一个活的文档。 每次 AI 犯错你纠正它时，以"编辑 CLAUDE.md 这样你不会再犯那个错误"结束。Claude 很擅长为自己写规则。随着时间，你的 CLAUDE.md 成为自我改进的规则手册。错误率可测量地下降，因为 AI 字面上在编码它自己的纠正（理想情况下你想发生）。

用 lessons.md 文件进一步。每次纠正后，每次 PR 后，每次调试会话后，Claude 用导致问题的模式和防止它的规则更新 lessons.md。让你的 CLAUDE.md 指向它："会话开始时审查 lessons.md 获取相关项目。"现在 AI 从它自己在你项目上的历史学习。这是把好的 CLAUDE.md 和伟大的分开的自我改进循环。

示例 CLAUDE.md：

```
技术栈：Next.js 14、TypeScript、Tailwind CSS、Supabase
所有组件放在 src/components/
使用带 hooks 的功能组件
所有 API 路由放在 src/app/api/
永远不要使用内联样式。总是用 Tailwind。
```

```
设计令牌：主蓝 #3B82F6、背景 #F9FAFB
Mobile-first 响应式方法
参考文档：PRD.md、APP_FLOW.md、TECH_STACK.md、FRONTEND_GUIDELINES.md、BACKEND_STRUCTURE.md、IMPLEMENTATION_PLAN.md
每次会话开始读取 progress.txt。完成任何功能后更新 progress.txt。会话开始时审查 lessons.md。每次纠正后更新它。
```

当 Claude Code 读取这个时，它在创建的每个文件中都遵循这些规则，没有漂移，没有随机决策。纯粹的Consistency。

Cursor 有它自己的版本：`.cursor/rules` 文件。相同概念，不同工具。把你的项目规则放在 `.cursor/rules/` 里，Cursor 在所有模式下自动读取它们。如果你在同一个项目上同时使用 Cursor 和 Claude Code，保持 CLAUDE.md 和 Cursor 规则对齐。相同的约束、相同的约定、相同的真相来源。有些人维护一个并复制到另一个。有些人写一个共享规则文件并从两者引用。都可以。关键是每个接触你代码库的 AI 工具都遵循相同的规则。

progress.txt 是你的会话桥。在每次功能后更新它：

```
已完成：
通过 Clerk 的用户认证（登录、注册、Google OAuth）
带侧边栏导航的仪表盘布局
产品 API 端点（GET /api/products）
进行中：
产品详情页面（/products/[id]）
需要连接 frontend 到 API
接下来：
购物车功能
Stripe 结账
已知 Bug：
点击链接后手机导航不关闭
```

当你打开新终端、开始新 Claude Code 会话、切换分支，或一周后回来时，AI 读取这个并确切知道你在哪里。没有"我们在做什么？"没有从零开始重建上下文。没有第 69 次重新解释你的整个项目。

这就是系统。 规范文档定义构建什么。CLAUDE.md 强制执行规则。Progress.txt 在会话间保持状态。一起它们给 AI 构建你项目所需的一切，不产生幻觉。

你拥有的 Markdown 文档越多，AI 猜测越少。AI 猜测越少，你就越好。

十八、工具以及如何使用它们

你不需要 50 个工具。你需要这些。但你需要正确地使用它们。不同阶段用不同工具。

Cursor - 你的代码编辑器。AI 原生，为这种工作流程构建。它看到你的整个项目，理解文件关系，能同时编辑多个文件。

但 Cursor 不只是一样东西。它有四种模式，大多数人只用一种：

Ask mode 是只读的。AI 读取你的代码库并回答问题而不改变任何东西。当你探索不熟悉的代码、试图理解某事如何工作，或在碰文件前规划下一步时使用它。把它当作你的代码顾问。"这个函数做什么？""为什么这个组件重新渲染？""如果我改这个模式会打破什么？"Ask mode 是你行动前思考的地方。

Plan mode 是你编码前架构的地方。你描述你想构建什么，Cursor 创建详细的实施计划与步骤，问你澄清问题，并能生成方法的视觉图表。在每个新功能开始时使用它。"我需要给这个 App 加 Stripe 结账。创建一个计划。"审查计划，调整它，批准步骤。然后把那些步骤发给 Agent mode 执行。

Agent mode 是主力。这是 AI 自主写代码、编辑文件、运行终端命令、安装包、修复错误的地方。它读取你的整个项目，遵循你的规则文件，端到端实现功能。当你说"构建仪表盘页面"时，就是这个模式在做工作。大多数氛围编程发生在这里。

Debug mode 是人们不知道的那个。当你遇到顽固的 bug 时，Debug mode 不只是尝试随机修复。它用运行时日志给你的代码插桩，生成关于什么出错的多个假设，让你复现 bug，测试它的修复，并让你验证。它是内置于编辑器的结构化调试循环。

Cursor 内部的工作流程：Ask 理解 → Plan 架构 → Agent 构建 → Debug 当它崩了。

Claude - 通过 Claude.ai 或 Claude Code。Claude 是你的思考伙伴。用 Claude 做繁重的思考：审问你的想法、写你的六份规范文档、规划架构、处理复杂的产品决策、起草你的 CLAUDE.md。Claude 是你做询问、规划、写作的地方，这些后来喂养其他一切。

Claude Code 在你的终端运行并自动从项目根目录读取 CLAUDE.md。它遵循你的规则而不需要你重复它们。对于更大的重构、文档繁重的任务、多文件架构变更，Claude Code 是你最好的工具。

Kimi K2.5 - Moonshot AI 的开源视觉编码模型。这是 frontend 实施的专家。K2.5 是一个原生多模态模型，意味着它从一开始就一起接受视觉和文本训练。你可以喂它截图、视频或设计模型，它生成功能性的 frontend 代码，紧密匹配视觉。布局、动画、交互、响应行为。其他模型近似你的设计，K2.5 复制它。当你把设计翻译成代码时使用 Kimi K2.5（通过 Kimi Code 或在 Cursor 中用模型选择器）。你有定义系统的 FRONTEND_GUIDELINES.md，你有参考的截图，你需要像素级精确的实施。这就是 K2.5 的赛道。

Codex - OpenAI 的编码代理。这是你的调试器和终结者。Codex 在云端、通过 Codex CLI 在你的终端，或直接在你的 IDE 中运行。每个任务得到自己的预加载你仓库的隔离沙盒。让 Codex 不同的是它如何处理调试：它读取你的代码库、跟踪依赖、审查配置、跨文件提出修复。它可以运行你的测试、修复失败、迭代直到全部通过。

在文件和架构构建完成后使用 Codex。当结构就位但东西在崩、当你需要在发布前代码审查、当你想重构而不破坏现有功能、当你需要有人找到你漏掉的 bug。Codex 是终结者。你也可以并行运行多个 Codex 任务在不同的 bug 或功能上。它异步工作。启动任务，去做别的事，回来审查结果。

多工具工作流程：
Claude 做思考。Claude 写你的文档、规划你的架构、做产品决策。
Cursor Agent mode（或 Claude Code，或 Kimi K2.5）做构建。它从计划实施功能、生成组件、连接 frontend 到 backend。根据任务选择你利用的模型：

- K2.5 做视觉重的 frontend 工作
- Claude Code 做架构和文档繁重的工作
- Cursor Agent 做一般实施

Codex 做调试和完成。针对你构建的代码库运行它。让它找到 bug、跟踪失败、审查你的代码、提出修复。让它运行测试直到通过。然后，干净地发布。
GitHub - 你的代码住在云端的地方，有版本控制和每次更改的跟踪。你可以回到任何以前的版本。如果你打破了什么，你可以撤销它。对于你的技术栈来说是不可协商的。

Vercel - 这是用于部署。推送到 GitHub，Vercel 自动构建和部署你的代码。你得到一个实时 URL，Vercel 上的免费层很慷慨。你的 App 在几分钟内从你的电脑到互联网上直播。

Supabase - 这是用于数据库和认证。他们有免费层，很容易设置。处理 backend 的东西，这样你可以专注于产品。

掌握这些工具。理解何时使用每一个。 ship 的人和卡住的人之间的区别不是他们用哪个 AI 模型，而是知道在适用时使用哪个 AI 模型。

十八点五：高级工作流程（当你准备好时）

一旦你有了基础系统构建了多个项目，这些技术会让你的速度倍增。这些直接来自为生计构建 Claude Code 的人。

用 git worktrees 的并行会话。 这是单一最大的生产力解锁。不是一次只做一件事，旋转 3 到 5 个 git worktrees，每个在自己的 Claude Code 会话中并行运行。一个 worktree 构建认证系统。另一个构建仪表盘布局。第三个处理 API 端点。它们都共享同一个仓库但独立工作。你在它们之间跳跃，审查输出，批准更改，每个完成时合并。过去需要一整天顺序构建的东西变成几个小时的并行执行。给你的 worktrees 命名并设置 shell 别名，这样你可以一键在它们之间跳跃。

Plan mode 作为你的力量倍增器。 你已经知道使用 Cursor 的 Plan mode。这里是如何让它更强大：让一个 Claude 会话写计划，然后旋转第二个会话告诉它作为高级工程师审查计划。"审查这个实施计划。找到 gaps、我漏掉的边缘情况、任何会打破的东西。"在写一行代

码之前修复计划。当实施期间出问题时，立即停止。不要继续推。切换回 plan mode 并从你所在的地方重新计划。也用 plan mode 做验证步骤，不只是构建。"规划如何验证这个认证流程处理所有边缘情况。"

复杂问题的子代理。 当问题大到单个 Claude 会话会在上下文上窒息时，使用子代理。追加"use subagents"到你的请求，Claude 会旋转专注的子会话来研究、探索、并行分析，同时保持你主会话的上下文窗口干净。每个子代理一个任务。把它当作委托给团队而不是自己做一切。

自定义技能和斜杠命令。 如果你一天做某事超过一次，把它变成可复用的技能或斜杠命令并提交到 git。构建一个 `/techdebt` 命令，在每个会话结束时运行，找到并杀死重复代码。构建一个 context-sync 命令，拉取最近一周的 Slack、文档、GitHub 活动到一个 dump，这样 Claude 每次会话都完全知情开始。技能是你教 Claude 针对你工作流程的新能力的方式。它们在你接触的每个项目上复合。

自主 bug 修复。 停止手把手调试。当你得到 bug 报告时，粘贴到 Claude 里说"修复"。当 CI 测试失败时，说"去修复失败的 CI 测试。"不要解释如何。指向日志、错误跟踪、失败测试给 Claude。让它跟踪问题、找到根本原因、解决它。你零上下文切换。Claude 令人惊讶地擅长阅读 docker 日志、跟踪分布式系统、手动修复你会花几个小时的东西。

语音听写以获得更好的提示词。 你说话比打字快三倍，你的提示词因此得到显著提高。在 macOS 上，按 fn 键两次开始听写。对话式描述你想要什么，带着你通常会跳过的大量细微差别和上下文，因为打字很慢。更长、更详细的提示词产生更好的输出。语音是到达那里的捷径。

学习模式。 当你想理解 AI 在做什么，不只是让它做时，在 Claude Code 的配置中启用"解释性"或"学习"输出风格。Claude 会解释它做的每个更改背后的推理。你也可以要求 Claude 生成视觉 HTML 演示来解释不熟悉的代码，绘制架构和协议的 ASCII 图表，或从新概念构建间隔重复抽认卡。边构建边学习的人是最终停止需要这篇指南的人。

十九、Git 和版本控制

Git 跟踪你做的每个更改。

没有 git：你打破什么就无法撤销，你失去跟踪什么改了，一个错误可以摧毁一切。

有 git：每次更改都保存，你可以回到任何以前的版本，你的代码住在 GitHub 上不只是你的电脑。

基础：
`git add .` 暂存你的更改。`git commit -m "message"` 用描述保存它们。`git push` 上传到 GitHub。`git pull` 下载最新更改。

当你在氛围编程时，经常提交。在每个有效的主要功能后："添加了用户登录"、"添加了产品网格"、"修复了结账 bug"。如果你打破什么，你总是可以回去。

这连接到你的 progress.txt 系统。提交代码，更新 progress.txt，推送两者。现在 GitHub 有你的代码和你的上下文文件。下次会话，拉下来，读 progress.txt，继续构建。

二十、环境变量和密钥

你的 API 密钥、数据库密码和密钥不应该在你的公共或生产代码库中。

它们放在 .env 文件里。

然后在你的代码中通过 `process.env.YOUR_KEY_NAME` 访问它们。

规则：永远不要提交 .env 到 git（加到 .gitignore）。永远不要在聊天中粘贴 API 密钥或用 AI 应用截图。如果你那样做，假设它已被泄露。永远不要在前端代码中放密钥，只在后端。开发和生产用不同的密钥。

当你部署到 Vercel 时，在 Vercel 的 dashboard 设置中添加你的环境变量。它们不会自动从你的本地 .env 转移。

如果你泄露了密钥，立即去那个服务撤销它。创建一个新的。

二十一、部署

你的代码在你的电脑上工作。现在它需要在互联网上工作。

Vercel 的流程：
推代码到 GitHub
连接 GitHub 仓库到 Vercel
Vercel 自动构建和部署
你得到一个像 your-app.vercel.app 的 URL
在 Vercel 的 dashboard 添加你的环境变量
当某些东西在本地工作但在部署时崩了，给 AI Vercel 错误日志。

"它在 localhost 上工作但在 Vercel 上崩了。”

这是来自 Vercel 日志的错误，粘贴错误给AI。

99% 的部署 bug 是缺少 ENV 变量或错误的构建设置。

二十二、阅读错误信息

错误不是侮辱，它们是指令。

错误信息告诉你确切什么出错了。大多数人恐慌并忽略它们。请不要那样做。

错误剖析：

```
TypeError: Cannot read property 'map' of undefined
    at ProductList (src/components/ProductList.tsx:15:23)
```

翻译：TypeError 意味着你用错了什么。"Cannot read property 'map' of undefined" 意味着你尝试在不存在的东西上用 .map()。`ProductList.tsx:15:23` 告诉你确切的文件和行号。

当你得到错误时，给 AI 一切：
"我得到这个错误：[完整错误信息]。这是那行的代码：[粘贴相关代码]"

你给 AI 的错误上下文越多，潜在的修复就越快。

二十三、调试循环

当某些东西坏了时：
读错误。真的读它。
找到位置。什么文件，什么行。
理解声明。错误说什么是错的?
检查明显的。拼写错误、缺失导入、错误的变量名。
给 AI 上下文。错误 + 代码 + 你期望什么。

循环：AI 给你代码 → 你尝试 → 它崩了 → 你粘贴错误 → AI 修复 → 重复直到工作。

对于在这个循环中存活两三轮的顽固 bug，切换工具。使用 Cursor 的 Debug mode。它生成多个假设，给你的代码插桩运行时日志，系统性地走过 bug 而不是猜测。或者启动 Codex。给它 bug 描述，让它跟踪你的代码库，识别跨文件的根本原因，迭代直到测试通过。Codex 特别擅长跨越多个文件或涉及数据流问题的 bug，因为它在提出修复之前读取整个仓库。

这是正常的。氛围编程不应该是通常的一次性方法，尽管我们喜欢那样看它。它是迭代。技能是快速迭代并知道何时使用哪个工具，不是完全避免迭代。

二十四、发布前验证

发布前，检查：
它在手机上工作吗？真的在你手机上打开它。
它在不同浏览器中工作吗?
没有数据时发生什么，空状态处理了吗?

错误数据时发生什么，错误状态处理了吗?
慢网速时发生什么，加载状态存在吗?
你能通过快速点击打破它吗?
密钥在浏览器开发者工具中隐藏了吗?

直到你回答了这些再发布。你的用户会找到你漏掉的每个 bug。

一丝不苟地做这个过程。

二十五、跟 AI 谈论所有这些

现在你有词汇表了...你需要使用它。

模糊提示：
"给我构建一个用户可以发帖的 App"

有文档支持的特定提示：
"首先读取 CLAUDE.md 和 progress.txt。然后构建 IMPLEMENTATION_PLAN.md 的步骤 4.2。登录流程在 APP_FLOW.md 第 3 节定义。使用来自 BACKEND_STRUCTURE.md 第 5 节的认证设置。按照 FRONTEND_GUIDELINES.md 样式化一切。匹配附加截图的 UI。"

同样的想法。完全不同的输出质量。

特异性不是额外工作，它是"工作"。你预先定义的越多，后来调试的越少。

二十六、如何阅读 AI 的输出

AI 给你一些代码。你知道你在看什么吗?

你不需要理解每一行。但你需要理解结构。创建了什么文件？它们做什么？它们如何连接?

当 AI 生成代码时，问这个："用 plain English 解释你刚构建了什么。每个文件做什么？它们如何连接？"

随着时间，你会开始识别模式。你会看到一个 import 语句并知道它在拉另一个文件。你会看到一个 useState 并知道它在跟踪变化的东西。你会看到一个 API 调用并知道它在获取数据。

这就是你从氛围编码者变成真正构建者的方式。不是通过记忆语法，而是通过理解模式。

二十七、如何迭代

每个人的第一次输出都很少是对的...这没问题。

迭代系统：
AI 构建版本 1
你测试它，找到什么错了
你具体描述什么错了（不是"它坏了"，而是"提交按钮不保存到数据库，这是错误"）
AI 修复它
你再测试
重复

好的迭代："产品网格在桌面上显示 4 列但我需要 3 列。卡片图片拉伸了，它们应该是 object-cover。数据获取时没有加载状态。"

坏的迭代："它看起来不对，修复它。"

要具体。一以贯之。

二十八、把大想法拆成小碎片

AI 在大而高的请求上窒息。

"给我构建一个完整的电商网站"可能会产生绝对的垃圾。

拆成碎片：
设置项目脚手架和安装依赖
构建导航栏组件
构建产品卡片组件
构建产品网格页面
连接数据库并获取产品
构建单个产品页面
加入购物车功能
构建购物车页面
Stripe 结账

每个碎片是一个对话或一个任务。

每个建立在上一个之上，每个可以独立测试。把自己带回 LEGO 日子。实际上就是这样。

这字面上就是你的 IMPLEMENTATION_PLAN.md 是什么。上面那个列表，编号和排序，正是你实施计划的精确格式。你不是在编码时即时创建它。你在开始前就创建了它。现在你只是执行它。逐步有计划地。

告诉 AI："构建 IMPLEMENTATION_PLAN.md 的步骤 5。

"不是"构建下一个东西"。精度复合。

这连接回你的 progress.txt 文件。在每个碎片后，更新 progress。用新鲜上下文开始下一个碎片。

二十九、什么时候 AI 是错的工具

有时候你只需要学习这个东西。

用 AI 来：生成样板代码、写重复逻辑、快速探索方法、用上下文调试、把你的意图翻译成代码。

自己学习：核心概念（这篇指南里的一切）、如何阅读 AI 生成的代码、如何发现 AI 错了、如何在 AI 帮不了时调试、你选择的栈如何基础工作。

如果你依赖 AI 做一切，你是在流沙上构建。一个奇怪的 bug 你会卡住并持续解决不了。

如果 AI 解释错了什么而你相信它，嗯...你完蛋了。

所以也许，只是也许现在学习这些东西。

另外：很多时候网站文档比 AI 信息更好。Stack Overflow 有对确切错误的精确答案。文档网站有权威信息。AI 擅长合成和生成，但当它失败时，知道如何搜索官方文档和网站是没人谈论的 fallback 技能。Claude 甚至可以从这些自学它需要的一切。

三十、范围和知道何时停止

一个永无止境的功能列表杀死比一些坏代码更多的项目。

当你完成时：
核心功能工作，用户可以完成主要行动，在常见路径上不崩，它已部署并可访问。

当你还没完成时：
它是"完美的"，每个边缘情况都处理了，它有你想象的每个功能，或它看起来完全像 Dribbble 设计。

发布简单版本并获取你的反馈。然后，基于真实使用改进。

你从未发布的最好的 App 比你已上线的普通 App 更糟。

三十一、维护你构建的东西

你构建了它，现在你需要改变它：

未来的你（或 AI）需要理解代码。这就是你的文档系统真正回报的地方。

你的 README 解释项目是什么。你的 CLAUDE.md 强制执行规则。你的 progress.txt 显示构建了什么和接下来是什么。你的规范 Markdown 文档定义产品的每个方面，从需求到实施顺序。

当你 3 个月后回到代码时：
"阅读 CLAUDE.md、progress.txt 和 PRD.md。我在休息后回到这个项目。根据 IMPLEMENTATION_PLAN.md 总结事情在哪里以及需要注意什么。"

好的文档让未来的氛围编码会话快 10 倍。坏的文档意味着从零开始。

保持依赖更新、在令人困惑的部分写注释、使用一致的命名、把你的代码库当作一个陌生人在 Airbnb 会访问并需要知道东西在哪里的家。

三十二、成本意识

API 调用花钱。数据库花钱。托管可能花钱。AI 工具花钱。

免费层存在而且它们很慷慨。Vercel 免费，Supabase 免费，Clerk 免费（有限制）等等。

对于你工作流程中的 AI 工具：Cursor Pro 覆盖大多数模型访问，是你的主要订阅。Claude Code 使用通过你的 Anthropic 或 Claude 计划。Codex 包含在 ChatGPT Plus、Pro 和更高层中。Kimi K2.5 是开源的，通过 http://Kimi.com 免费访问，API 定价慷慨。

你不需要第一天就每个付费层。从 Cursor Pro 和 Claude 或 Gemini Pro 3 开始。当你的工作流程需要时添加其他的。

知道什么是免费的 vs 什么会累积。AI API 调用（OpenAI、Anthropic）按 token 花钱。超过免费层后数据库存储花钱。规模上的图片托管花钱。

从免费开始。需要时扩展。

不要为第一天就为数百万用户做架构。

因为现实是你可能根本不会达到那个目标。从小开始。

三十三、安全基础

最低要求：
永远不要在前端代码中暴露 API 密钥。总是在后端验证输入（不要相信来自浏览器的任何东西）。使用 HTTPS（Vercel 自动做这个）。保持依赖更新（过时的包有已知漏洞）。使用认证服务而不是自己构建。

你不是在构建一个高安全银行，但你也应该知道所有安全基础，这样你就不会带着明显漏洞发布你的 App。

也这样你就不会在互联网上被嘲笑。

大多数人的噩梦。

三十四、第三方服务

你不是从零构建一切。

你插入服务来处理困难的部分。

认证： Clerk、Supabase Auth。处理登录、注册、会话。

支付： Stripe。处理钱。

数据库： Supabase、Firebase。处理存储。

邮件： Resend、SendGrid。处理交易邮件。

文件上传： Uploadthing、Supabase Storage。处理媒体。

什么时候伸手拿服务：每当这个功能不是你的核心产品时。你的核心产品是你正在构建的独特东西。其他一切都是基础设施。

用现有服务做基础设施。

三十五、Assets、媒体和组件库

图片、字体、图标和预构建组件。

在哪里获取和找到它们，以及如何使用它们。

图标： Lucide React。免费、一致、易于使用。

字体： Google Fonts。在你的布局中导入，到处使用。

图片： Unsplash 用于库存。你自己的用于产品拍摄。AI 生成的用于定制。文件大小很重要：大图片减慢你的网站。上传前压缩。使用 Next.js Image 组件自动优化。

组件库是大多数初学者不知道的作弊码。不是从零构建每个按钮、模态框、下拉菜单和表单元素，使用一个库给你开箱即用的精致、可访问、可定制组件。

shadcn/ui 是目前 Next.js 和 Tailwind 生态系统中的主导选择。它不是作为依赖安装的传统库。它直接把组件源代码复制到你的项目，意味着你拥有代码并可以定制一切。按钮、对话框、下拉菜单、标签、表单、数据表，全部建立在 Radix UI 原语之上，带 Tailwind 样式。告诉 AI："使用 shadcn/ui 组件。用 `npx shadcn@latest init` 初始化并添加我们需要的组件。"这节省数小时构建基础 UI 元素的时间，从一开始就给你可访问的、生产质量的组件。

在你的 TECH_STACK.md 和 FRONTEND_GUIDELINES.md 中注明你的组件库选择，这样 AI 在每个页面上都一致地使用它。

完整的系统

你现在拥有一切。

构建之前：
运行审问提示词。让 AI 残酷地审问你的想法。
回答 AI 问的每个问题。
使用文档生成提示词创建你的六份规范文档：PRD.md、APP_FLOW.md、TECH_STACK.md、FRONTEND_GUIDELINES.md、BACKEND_STRUCTURE.md、IMPLEMENTATION_PLAN.md
用项目规则、所有六份文档的引用、和 lessons.md 自我改进循环写你的 CLAUDE.md
用你起始状态创建 progress.txt
收集 UI 截图作为参考
初始化 git 并推送到 GitHub

构建期间：
AI 每次会话首先读取 CLAUDE.md、progress.txt 和 lessons.md
在 Cursor（或 Claude）中使用 Ask 和 Plan 模式在编码前做架构
使用 Agent 模式、Claude Code 或 Kimi K2.5 实施（根据任务匹配工具）
小碎片工作，一次一个功能
给引用你规范文档的具体的、词汇丰富的提示词
截图参考用于 UI 工作
每个有效功能后提交到 git
每个功能后更新 progress.txt
每次纠正后更新 CLAUDE.md 和 lessons.md，这样 AI 永远不会重复同样的错误
架构就位后使用 Codex 调试、审查和完成

定期在手机上测试
阅读错误，不要恐慌

发布之前：
检查手机
检查错误状态和空状态
验证密钥隐藏
从头到尾测试主要用户流程
检查性能（有什么慢的吗？）

发布之后：
更新文档反映构建了什么
定期更新依赖
基于真实用户反馈迭代
保持 progress.txt 和 lessons.md 最新用于未来会话
把重复的工作流程变成可复用的技能和斜杠命令

这就是完整的系统。

氛围编程不是巫术黑魔法。所有这些都是细致的计划、系统、文档、词汇和迭代。你审问你的想法。你写你的 Markdown 文档。你设置 CLAUDE.md、progress.txt 和 lessons.md 用于持久性和自我改进。你为每个阶段使用正确的工具：Claude 用于询问和思考，Cursor 模式用于构建，Kimi K2.5 用于视觉实施，Codex 用于调试和完成。你用具体的术语描述工作。你在会话间跟踪你的进度。你提交你的代码。然后你发布。

AI 现在做所有的打字。
你做所有的思考。

现在你没有借口了。

今天就去构建点该死的东西。

如果你看到这里，你已经领先 99% 会收藏这篇然后保证永远不会回来看的人。比他们更好。

注：我不是传统开发者。我是自学的。这篇指南中的一切都来自艰难地构建东西、打破它们、弄清楚为什么，并写下实际有效的东西。如果有什么错了、缺失或过时了，告诉我。这是一份活的文档，我宁愿修复它也不愿让某人基于坏建议构建。